TALLER PRÁCTICO

Backend para tu Startup

Guía Metodológica | Visualización 2 (VIS2-2026I)

Julián Zuluaga — Docente

20 de marzo de 2026

Introducción

Esta guía te lleva paso a paso desde tu frontend desplegado en Vercel hasta tener un **backend funcional conectado a Supabase** — todo desde la terminal, usando tu agente CLI.

Ya completaste las fases de diseño conceptual, documentación de negocio, GitHub, landing page, diseño en Figma y maquetación del frontend. Ahora viene el momento de darle vida a tu app con datos reales.

Lo que ya tienes

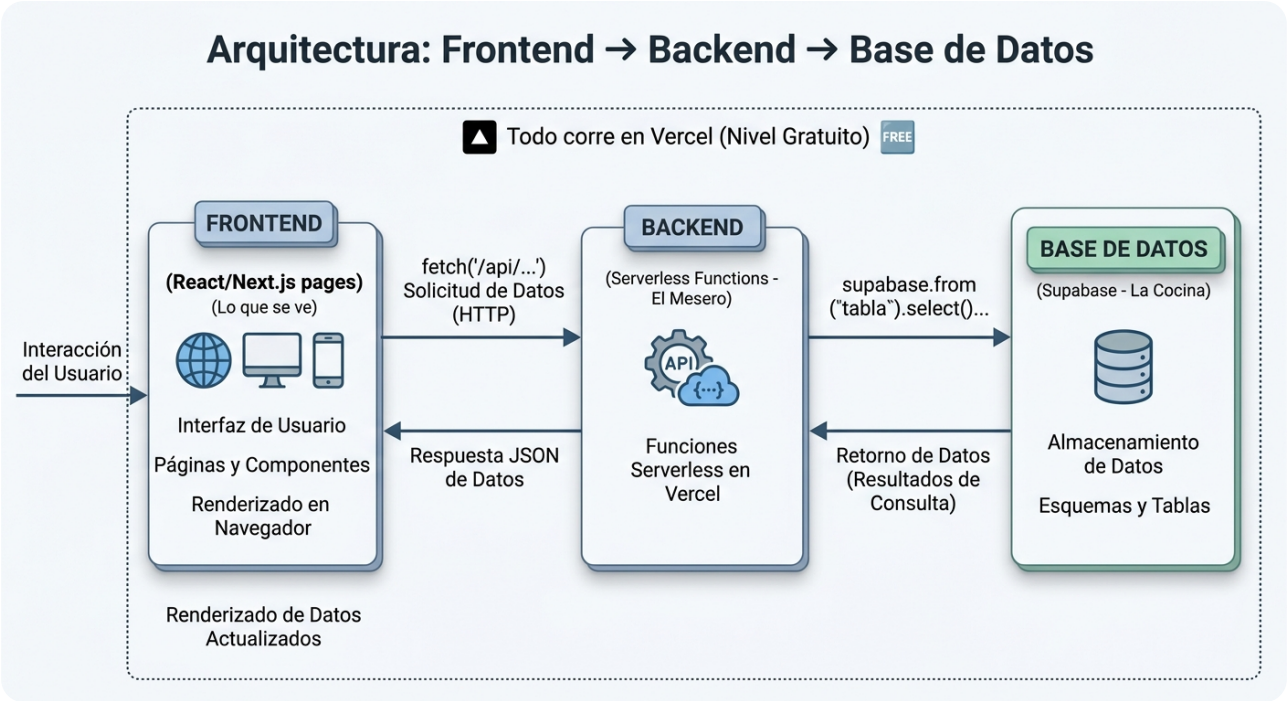
Recurso	Estado
Modelo de negocio	`docs/negocio/modelo_de_negocio.md`
Flujos de usuario	`docs/negocio/flujos_de_usuario.md`
Esquema de BD	Supabase (tablas creadas)
Frontend	Desplegado en Vercel (Figma → HTML/CSS/JS)
Stack técnico	`docs/arquitectura/stack.md`
Supabase MCP	Conectado al agente CLI
Vercel CLI	Instalado y linkeado

Lo que vas a construir

API Routes en Next.js — archivos dentro de `app/api/` que actúan como funciones serverless. Son tu "backend" sin necesidad de montar un servidor.

---

1. La Arquitectura



Arquitectura de 3 capas: Frontend (lo que se ve) → Backend/API Routes (el mesero) → Supabase (la cocina)

Tu aplicación tiene 3 capas:

Frontend (React/Next.js) — Las páginas que el usuario ve e interactúa. Ya lo tienes desplegado.

Backend (API Routes) — Funciones serverless en `app/api/`. Reciben peticiones del frontend, consultan Supabase, y devuelven datos en formato JSON. Es el mesero del restaurante: lleva el pedido a la cocina y trae la comida.

Base de Datos (Supabase) — PostgreSQL en la nube. Ya tienes las tablas creadas. Es la cocina donde están los datos.

Todo corre en **Vercel gratuito**. No necesitas otro hosting.

---

2. Tu Toolkit de Terminal

Las 3 herramientas que usarás desde la terminal para construir y desplegar tu backend

Tienes 3 herramientas que trabajan juntas desde tu terminal:

Agente CLI (Claude, Cursor, etc.)

- **Lee** tu código y entiende tu proyecto
- **Escribe** archivos y genera endpoints
- **Explica** errores y sugiere soluciones
- **Genera** la arquitectura backend completa

Supabase MCP (conexión directa a la BD)

- **Consulta** tablas y columnas sin abrir el browser
- **Ejecuta** SQL para verificar datos
- **Aplica** migraciones si necesitas cambiar el esquema

Vercel CLI (deploy desde terminal)

- `vercel env add` — Configura variables de entorno
- `vercel` — Deploy de prueba (URL temporal)
- `vercel --prod` — Deploy a producción

---

3. Configuración Inicial

Antes de escribir código, prepara el entorno. Todo desde terminal:

3.1 Verificar herramientas

```
vercel --version        # Verificar Vercel CLI
vercel whoami           # Verificar login
node --version          # Verificar Node.js
npm --version           # Verificar npm
```

3.2 Variables de entorno (desde terminal)

Necesitas 3 valores de Supabase. Pídele a tu agente: "Dame la URL y las keys de mi proyecto de Supabase".

```
# Agregar las 3 variables a Vercel
vercel env add NEXT_PUBLIC_SUPABASE_URL
vercel env add NEXT_PUBLIC_SUPABASE_ANON_KEY
vercel env add SUPABASE_SERVICE_ROLE_KEY

# Bajar a tu máquina (crea .env.local automáticamente)
vercel env pull .env.local

# Verificar que están
vercel env ls
```

> Cuando pregunte el ambiente, selecciona **Production, Preview y Development** (los 3).

3.3 Seguridad

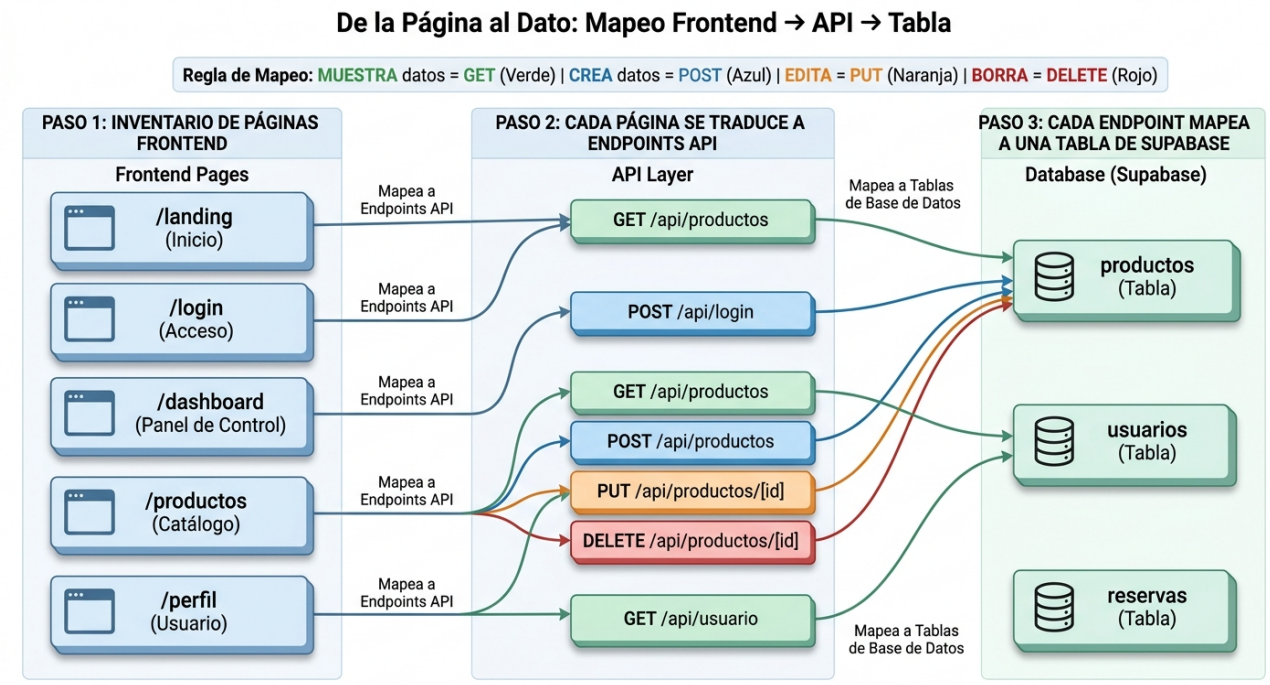
```
# Verificar que .env.local está en .gitignore
grep ".env.local" .gitignore

# Si NO aparece, agregarlo (CRÍTICO)
echo ".env.local" >> .gitignore
```

3.4 Instalar Supabase SDK

```
npm install @supabase/supabase-js
```

4. De la Página al Dato



Cada página del frontend se traduce a endpoints API, y cada endpoint mapea a una tabla de Supabase

Este es el ejercicio más importante: traducir lo visual a lo técnico.

Paso 1: Inventario de Páginas

Lista TODAS las páginas de tu frontend y responde: ¿qué muestra? ¿el usuario puede crear, editar o borrar algo?

Página	¿Qué muestra?	¿Acción del usuario?
`/`	Landing page	No — solo informativa
`/login`	Formulario	Envía email y contraseña
`/dashboard`	Panel resumen	Muestra datos del usuario
`/productos`	Lista	Puede filtrar y buscar
`/productos/nuevo`	Formulario	Crea un producto

Paso 2: Traducir a Endpoints API

La regla es simple:

La página...	Necesita un...
MUESTRA datos	`GET`
CREA datos	`POST`
EDITA datos	`PUT`
BORRA datos	`DELETE`

Paso 3: Mapear a Tablas

Cada endpoint toca una o más tablas. Pídele a tu agente: "lista las tablas de mi Supabase" — él tiene acceso por MCP.

Endpoint	Método	Tabla(s)	Qué hace
`/api/productos`	GET	`productos`	`select('*')` con filtros
`/api/productos`	POST	`productos`	`insert({...})`
`/api/productos/[id]`	PUT	`productos`	`update({...}).eq('id', id)`
`/api/productos/[id]`	DELETE	`productos`	`delete().eq('id', id)`

---

5. Estructura de Archivos

El archivo de conexión (`lib/supabase.ts`)

Se crea **una sola vez**. Es el puente entre tu código y la base de datos:

```
import { createClient } from '@supabase/supabase-js'

// Para el FRONTEND (navegador) — acceso limitado
export const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
)

// Para API ROUTES (servidor) — acceso total
export const supabaseAdmin = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!
)
```

Estructura de la carpeta `app/api/`

Un archivo `route.ts` por cada recurso (tabla principal):

```
app/
└── api/
    ├── productos/
    │   ├── route.ts          ← GET + POST
    │   └── [id]/
    │       └── route.ts      ← GET + PUT + DELETE
    ├── usuarios/
    │   └── route.ts
    └── auth/
        ├── login/
        │   └── route.ts      ← POST
        └── register/
            └── route.ts      ← POST
```

---

6. Anatomía de un Endpoint

Todos tus endpoints siguen el **mismo patrón**. Aprende uno, aplícalo a todos:

GET — Leer datos

```
import { supabaseAdmin } from '@/lib/supabase'
import { NextRequest, NextResponse } from 'next/server'

export async function GET(request: NextRequest) {
  const { searchParams } = new URL(request.url)
  const categoria = searchParams.get('categoria')

  let query = supabaseAdmin.from('productos').select('*')
  if (categoria) {
    query = query.eq('categoria', categoria)
  }
  const { data, error } = await query

  if (error) {
    return NextResponse.json(
      { error: error.message }, { status: 500 }
    )
  }
  return NextResponse.json(data)
}
```

POST — Crear datos

```
export async function POST(request: NextRequest) {
  const body = await request.json()

  if (!body.name || !body.price) {
    return NextResponse.json(
      { error: 'Nombre y precio son obligatorios' },
      { status: 400 }
    )
  }

  const { data, error } = await supabaseAdmin
    .from('productos')
    .insert({ name: body.name, price: body.price })
    .select()
    .single()

  if (error) {
    return NextResponse.json(
      { error: error.message }, { status: 400 }
    )
  }
  return NextResponse.json(data, { status: 201 })
}
```

Conectar desde el Frontend

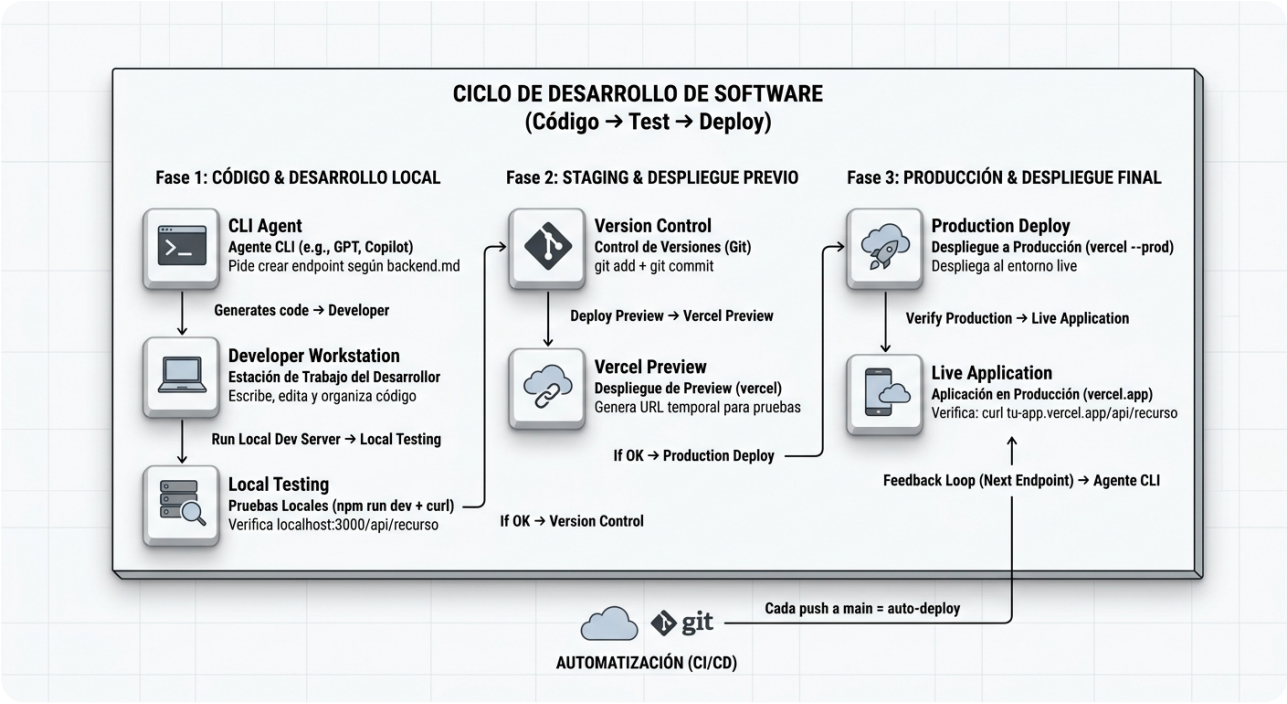
```
// ANTES (datos quemados)
const productos = [
  { id: 1, name: "Café", price: 5000 },
]

// DESPUÉS (datos reales)
const response = await fetch('/api/productos')
const productos = await response.json()

// Para CREAR
const res = await fetch('/api/productos', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ name: 'Café', price: 8000 })
})
```

---

7. Ciclo de Desarrollo



El ciclo completo: desde pedirle al agente CLI que cree el endpoint hasta verificar en producción

El loop de trabajo

```
# 1. Pídele a tu agente que cree un endpoint
> "Crea el endpoint GET /api/productos según backend.md"

# 2. Prueba localmente
npm run dev
curl http://localhost:3000/api/productos

# 3. Si funciona → commitea
git add app/api/productos/route.ts
git commit -m "feat: endpoint GET /api/productos"

# 4. Deploy preview (URL temporal)
vercel

# 5. Todo OK → producción
vercel --prod

# 6. Verificar en producción
curl https://tu-app.vercel.app/api/productos

# 7. Repetir con el siguiente endpoint
```

Cada `git push` a `main` también hace deploy automático.

---

8. Límites de Vercel Gratuito

Lo que SÍ puedes hacer

Recurso	Límite
API Routes	Ilimitadas
Tiempo por función	10 segundos
Deploys por día	100
Bandwidth	100 GB/mes
Preview deploys	Ilimitados
HTTPS	Automático

Lo que NO puedes hacer

Restricción	Alternativa
Funciones > 10s	Dividir la lógica
WebSockets	Supabase Realtime
Cron jobs	No aplica
Guardar archivos en servidor	Supabase Storage

---

9. El Prompt para tu Agente CLI

Una vez completes los pasos 1-4 (inventario, mapeo, tablas, estructura), dale este prompt a tu agente CLI:

> "Necesito que generes el documento `docs/arquitectura/backend.md`. Lee mis flujos de usuario (`docs/negocio/flujos_de_usuario.md`), consulta mis tablas de Supabase por MCP, revisa las páginas en `app/`, y genera: mapa de endpoints, estructura de archivos, código por endpoint, conexiones frontend→backend, validaciones, y orden de implementación."

El agente tiene acceso directo a todo — lee tu código, consulta la BD, y genera la arquitectura específica a tu startup.

Después, pídele que construya endpoint por endpoint:

> "Crea el primer endpoint del mapa: GET /api/[tu-recurso]"

---

10. Checklist Final

```
# Verificar desde terminal:
vercel env ls              # ¿3 variables configuradas?
npm run dev                # ¿Corre localmente?
curl localhost:3000/api/X  # ¿Endpoint responde?
vercel --prod              # ¿Deploy exitoso?
grep ".env.local" .gitignore  # ¿Keys protegidas?
```

Entregables

- [ ] `lib/supabase.ts` — Conexión a la BD
- [ ] `.env.local` — Generado con `vercel env pull` (NO commiteado)
- [ ] Variables en Vercel — `vercel env ls` las muestra
- [ ] Al menos 1 endpoint `GET` funcionando
- [ ] Al menos 1 endpoint `POST` funcionando
- [ ] Frontend con datos reales (no quemados)
- [ ] Deploy funcional en Vercel — `vercel --prod`
- [ ] `docs/arquitectura/backend.md` — Documentación de endpoints

Referencia Rápida de Comandos

Quiero...	Comando
Correr en local	`npm run dev`
Probar endpoint	`curl http://localhost:3000/api/recurso`
Ver variables Vercel	`vercel env ls`
Agregar variable	`vercel env add NOMBRE`
Bajar variables	`vercel env pull .env.local`
Deploy prueba	`vercel`
Deploy producción	`vercel --prod`
Ver deploys	`vercel ls`
Ver logs errores	`vercel logs URL`
Instalar paquete	`npm install paquete`
Consultar BD	"Lista tablas de Supabase" (agente CLI)
Ver columnas	"Columnas de la tabla X" (agente CLI)